Kutty Banerjee

kutty.banerjee@gmail.com (925)-209-7204 (US Citizen)

Summary

Experienced in Comp Architecture, System Software, Compilers, Simulators, System Wide Optimizations, Leading Cross-Team efforts across SW-HW boundaries.

Experience

Apple: iOS GPU Engineer [Aug 2013 – Present]

- Added in Firmware, a Multi Linear Regression based Model for predicting Runtime GPU Power Consumption.
- Augmented Firmware with a Credit based Policy for Fairness in GPU Work Scheduling.
- Added GPU Compiler and Driver support to implement Efficient Vertex Clipping for User Planes (OpenGL)
- Added Data flow logic in Driver to Elide Cache Flushes between OpenCL kernels missing RAW/WAW dependencies

Intel: GPU Platform Architect [May 2013 – Aug 2013]

- Drive Low Power design of Compute features on next-gen Intel GPUs. Use models to predict Perf and Power.

NVIDIA: Software Architect [May 2005 – May 2013]

Project Denver (NVIDIA's ARM64 CPU)

- Lead Android Stack optimizations for Denver CPU targeting Kernel, Browser JITs, Dalvik and coordinated with the required teams.
- Did CPU Perf Analysis of Graphics, Web and JVM workloads and coordinated with various HW and SW teams.
- Helped triage CPU Performance problems due to
  - Instruction Scheduling Stalls, Cache Bottlenecks, False Sharing
  - Missing Speculative Optimizations, Excessive Rollback of Instructions
- Implemented a **Memory Profiler** and augmented CPU/GPU **Functional Simulators** to use it.
  - Cost an additional 15-20% overhead at runtime.
  - Captured Physical/Virtual Pages accessed from various threads, cores along with Page attributes.
  - Data mined this using a Database offline to Graph, Page Access Patterns, Page-Migration Patterns for CPU/GPU workloads.
  - Added support to Profile and Graph Self Modifying Pages and common Pages mapped across Separate Address Spaces to better study L1 Cache aliasing effect over large applications.
- Denver uses a Trace JIT compiler to convert from ARM to native VLIW. I maintained the Trace Profiler and Feedback codebase. As part of that role implemented:
  - Branch Assert instruction to remove cold branches and improve Instruction Scheduling
  - Lazy Branch Assert to avoid recompiling traces for occasionally taken cold branches
  - Branch Re-Profiling to detect dynamic changes in branch weights of a running program and trigger recompilation. Libquantum showed 60% improvement.
  - Elide Fence instructions and allow reordering past fence instructions and provide a rollback if during commit another core accessed the speculative lines. Helps JVM workloads by making atomics noops.
- Added **Large Page Support in Linux kernel** using a BSD-like Lazy Page Coalescing approach for ARMv7.
- Improved Multi Page Size Support for ARMv8 in **CPU Simulator**
- Added "**Thread Local Heap"** support in Linux kernel to elide TLB invalidates, IPIs and cache snoop for private memory. Boosted Google Chrome and Mozilla by ~20% on the V8 benchmark.
- Worked with various Driver teams to enforce code correctness for ARM Weak Consistency Rules and AXI Bus ordering rules.

Project Orion (CUDA) – Cache Coherent CPU/GPU on a single SoC

- Was responsible for delivering UVM (Unified Virtual Memory) across CPU and GPU on a Custom NVIDIA SoC with an internal CPU GPU and Cache-Coherent Interconnect for the CUDA HPC Market.
- My role was to drive adding features in GPU, SoC and the Driver stack to make this happen. This included adding support for GPU Page Fault handling, GPU TLB redesign, Page Migration logic, interrupt coalescing and low latency interrupt delivery.
- Designed ARM64 Linux kernel changes to make this possible. Other engineers did the implementation.
- Implemented GPU Functional and Perf Simulator changes to test Performance impact of False-sharing, Page Thrashing, GPU TLB invalidate performance.
- Project was later cancelled but the above worked move to a different project

Parallel Computing Framework:

- During Sabbatical, implemented "**Khronos = A SW implementation of an Out-Of-Order processor**"
  - It's a Stack Based Virtual Machine with its own Instruction Set. Instructions are very similar to OpenCL kernels and Operands are Tiled Regions of Memory.
  - Instruction Parallelism is obtained by Data Flow analysis of Instructions forming an Anti-Dependency Graph which is then parsed by separate CPU Threads in Parallel to obtain Parallel execution for independent instructions.
  - Supports Branches via Predication and has first class Instructions for loops. Also has a special instruction for Parallel Reduction.
  - Added a Data-Aware Scheduler to factor in memory traffic induced by migrating work to GPU.
- Implemented a Mail Box based Thread Pool for Intra Thread communication in Khronos
  - Threads communicate with each other using dedicated point to point mail-boxes each with separate locks to minimize lock contention..
  - Implemented Biased Lock Support for the above. Biased locks acquire the lock without an atomic/fence instruction
- Implemented various Atomics/Fence free/Transactional algorithms.
- Implemented Reader lock with CAS-less/fence-less reader to minimize cache thrashing.

GPU Test Infrastructure Lead

- Lead a Cross-Team effort to unify verification efforts of various Hardware and Software/Driver teams on the MDIAD-MODS Platform.
- Added "Probe Based Verification" to Policy-Manager, which is the layer responsible for introducing faults/events and boundary conditions that allow testing hard-to-hit conditions deterministically from an external script.
  - Probes are hooks in Drivers, which can be programmed at runtime using a script to wait for asynchronous events to occur. Allows an external script to force certain hard-race conditions to occur and ensure Driver works correctly.
  - Power Management Transition Module was the biggest consumer of this Verification Feature.
- Coordinated with Hardware and Software teams working on PCI-E, Display, Cache/TLB, Clocks, Core-Graphics and Video to beef up verification using the above Framework.
- Merged SW use cases with RTL HW unit Test Bench to get more directed coverage

Fermi Generation GPU (DX11 compatible) Kernel Lead

- As the Kernel Driver lead, **lead a team of 25 engineers in the Resource Manager Team** and oversaw development across Simulation, RTL, Emulation, Bringup and Product Release.
- Role included Task Assignment, Initial Triage, Interaction with Hardware and other Software teams, mentoring new Engineers and Code Review. Responsible for overall progress of the Kernel team for Fermi.

- Was the first engineer to start work on the Kernel/Firmware for Fermi. Wrote and helped large parts of the kernel code including Thread Scheduling, Interrupt Handling, Memory Management, Page Table Setup, Context Switch Support using Firmware, Cache mgmt., **TLB invalidate batching**.
- Augmented Functional Simulator's Cache and Memory modeling pipeline.

Fermi Command Buffer Scheduling Lead

- Lead the Fermi Command Buffer Scheduling unit in the Driver. Fermi added Hardware support for DX11 Per Node Command Buffers. Replaced the old Time Slice Based Scheduler with Per Node Queues.
- Added support to hold off Buffer submits (throttle) to enable Engine Power Gating
- Did plenty of code refactoring and wrote lots of tests. Scheduling module had around 90% branch Code Coverage!!

Resource Manager Graphics Driver

- Worked on various parts of the kernel driver, fixed bugs, did triaging.
- Implemented a **Non Stalling Lock Free Interrupt Tree** for routing DXG kernel notifiers without delay

Education

MS Computer Science, Worcester Polytechnic Institute, MA [2003 Aug – 2005 May]

- Thesis: Remote Execution of 3D Graphics for Mobile Systems.
  - Implemented a Memory and Power Model to Predict Consumption for Graphical Workloads running on Mobile Devices.
  - Recipient of WPI Institute Fellowship Award for outstanding research work. Publications in Journals and Conferences

BE Computer Science, University of Mumbai [1999 July – 2003 May]

Publications http://www.pseudoparallel.com/publications

Languages C / Java / Objective-C / C++ / SystemC / Rust (learning)